ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΕΧΝΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ ΚΑΙ ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ

Προχωρημένα Θέματα Βάσεων Δεδομένων

Εξαμηνιαία Εργασία

Χριστοδουλάκης Σταύρος
el20890

Γεωργιάδη Δάφνη
el20189

GitHub Repository

(https://github.com/DaphneGeorgiadi/Advanced-Topics-in-Database-Systems-NTUA)

Η εξαμηνιαία εργασία αυτή υλοποιήθηκε στα πλαίσια του μαθήματος "Προχωρημένα Θέματα Βάσεων Δεδομένων". Σκοπός της ήταν η αξιολόγηση εργαλείων διαχείρισης μεγάλου όγκου δεδομένων σε ένα κατανεμημένο περιβάλλον. Συγκεκριμένα μέσω του περιβάλλοντος AWS cloud με χρήση του προγραμματιστικού μοντέλου Apache Spark κάναμε, με την βοήθεια κατάλληλων queries, διαχείριση πολλών dataset.

Το βασικό dataset περιλαμβάνει δεδομένα καταγραφής εγκλημάτων για το Los Angeles από το 2010 μέχρι σήμερα (αρχές Δεκεμβρίου του 2024). Τα συμπληρωματικά είναι αρχεία μικρότερου όγκου που περιλαμβάνουν δεδομένα όπως, απογραφικά στοιχεία για το Los Angeles, στοιχεία σχετικά με το μέσο εισόδημα ανά νοικοκυριό και τον ταχυδρομικό κώδικα, στοιχεία σχετικά με την τοποθεσία των αστυνομικών τμημάτων στο Los Angeles και στοιχεία που περιέχουν τις πλήρες περιγραφές του φυλετικού προφίλ.

- ## Query 1

Υλοποιούμε το query αυτό χρησιμοποιώντας τα **DataFrame** και **RDD** APIs και το εκτελούμε με 4 Spark executors. Στο query αυτό ταξινομούνται σε φθίνουσα σειρά οι ηλικιακές ομάδες των θυμάτων σε περιστατικά που λαμβάνουν οποιαδήποτε μορφή «βαριάς σωματικής βλάβης».
Η κατηγοριοποίηση είναι η ακόλουθη:

- Παιδιά: <18 και > 0
- Νεαροί ενήλικοι: 18–24
- Ενήλικοι: 25–64
- Ηλικιωμένοι:>64
- Unknown: όλα τα άλλα

Σημείωση: στο dataset υπάρχουν γραμμές όπου η ηλικία είναι μικρότερη ή ίση του μηδέν, για αυτό δημιουργήσαμε μία επιπλέον κατηγορία την Unknown.

Τα αρχικά βήματα είναι κοινά και για τα δύο APIs.
Στην αρχή φορτώσαμε τα δεδομένα για τα εγκλήματα από τα δύο .csv αρχεία και τα ενώσαμε για να τα έχουμε ομαδοποιημένα. Έπειτα ακολουθήθηκαν διαφορετικά βήματα με την ίδια λογική.

Χρησιμοποιώντας DataFrame:
Αποθηκεύουμε τον τρέχοντα χρόνο για να υπολογίσουμε τη διάρκεια εκτέλεσης του κώδικα. Ύστερα φιλτράραμε τα δεδομένα για να μείνουν μόνο εκείνα όπου στην στήλη "Crm Cd Desc" έχουν την τιμή "AGGRAVATED ASSAULT". Στην συνέχεια δημιουργήσαμε μία επιπλέον στήλη στο DataFrame η οποία ονομάζεται "Victim_Age_Group" και βασίζεται στην ηλικία του θύματος δεδομένου των παραπάνω κατηγοριών. Έπειτα χρησιμοποιούμε την μέθοδο "groupBy" για να ομαδοποιήσουμε τα δεδομένα με βάση την στήλη που δημιουργήσαμε και την μέθοδο "agg" που χρησιμοποιείται για να εφαρμοστεί η συνάρτηση "count" στις ομάδες που δημιουργήθηκαν. Επομένως το τελικό αποτέλεσμα αποθηκεύεται σε ένα νέο DataFrame "age_group_counts_df"με δύο στήλες ("Victim_Age_Group" και "Incident_Count"), κάνουμε sort σε αυτό το DataFrame και το εμφανίζουμε. Τέλος, υπολογίζουμε τον χρόνο εκτέλεσης που χρειάζεται για την εκτέλεση.

Τα αποτελέσματα είναι τα ακόλουθα:

```
+----------------+--------------+
|Victim_Age_Group|Incident_Count|
+----------------+--------------+
|          Adults|        121093|
|    Young Adults|         33605|
|        Children|         10830|
|         Elderly|          5985|
|         Unknown|          5098|
+----------------+--------------+

DataFrame Implementation Time: 4.30 seconds
```

Χρησιμοποιώντας RDD:
Αποθηκεύουμε τον τρέχοντα χρόνο για να υπολογίσουμε τη διάρκεια εκτέλεσης του κώδικα. Ύστερα δημιουργούμε ένα RDD από το DataFrame που περιέχει πληροφορίες για τα εγκλήματα, φιλτράροντας μόνο τις γραμμές που σχετίζονται με περιστατικά "AGGRAVATED ASSAULT". Μετά δημιουργούμε ένα καινούργιο RDD "age_group_rdd", όπου δημιουργεί ένα ζεύγος (tuple) όπου η πρώτη τιμή είναι η ηλικιακή ομάδα και η δεύτερη ο αριθμός 1. Έπειτα χρησιμοποιώντας την μέθοδο "reduceByKey" μετράμε το πλήθος των περιστατικών για κάθε ηλικιακή ομάδα. Τέλος κάνουμε ταξινομούμε και εμφανίζουμε τα αποτελέσματα. Τέλος, υπολογίζουμε τον χρόνο εκτέλεσης που χρειάζεται για την εκτέλεση.

Αποτελέσματα είναι τα ακόλουθα:

```
Adults: 121093
Young Adults: 33605
Children: 10830
Elderly: 5985
Unknown: 5098
RDD Implementation Time: 7.35 seconds
```

Από τους χρόνους που χρειάστηκαν φαίνονται τα ακόλουθα:

```
DataFrame Implementation Time: 4.30 seconds
RDD Implementation Time: 7.35 seconds
DataFrame API is faster.
```

Επομένως το DataFrame API είναι αρκετά πιο γρήγορο από το RDD API, παρόλο που και τα δύο έχουν χαμηλό χρόνο εκτέλεσης. Η διαφορά αυτή ήταν αναμενόμενη, αποδίδεται στο γεγονός ότι το DataFrame αξιοποιεί τον optimizer Spark's Catalyst για βελτιστοποίηση ερωτημάτων και τον Tungsten για αποτελεσματική εκτέλεση. Τα RDDs είναι συλλογές δεδομένων οι οποίες είναι ανθεκτικές σε σφάλματα και μπορούν να λειτουργούν παράλληλα, τα οποία όμως δεν έχουν από μόνα τους ενσωματωμένα optimization engine.

- ## Query 2

Υλοποιούμε το query αυτό χρησιμοποιώντας **DataFrame** και **SQL** APIs. Στο query αυτό βρίσκουμε για κάθε έτος τα τρία αστυνομικά τμήματα με το υψηλότερο ποσοστό κλεισμένων υποθέσεων και τυπώνουμε τα αποτελέσματα.
Τα αρχικά βήματα είναι κοινά και για τα δύο APIs.
Στην αρχή φορτώσαμε τα δεδομένα για τα εγκλήματα από τα δύο .csv αρχεία και τα ενώσαμε για καλύτερη διαχείριση. Έπειτα ακολουθήθηκαν διαφορετικά βήματα με την ίδια λογική.

Χρησιμοποιώντας DataFrame:
Αποθηκεύουμε τον τρέχοντα χρόνο για να υπολογίσουμε τη διάρκεια εκτέλεσης του κώδικα. Δημιουργούμε μια νέα στήλη στο DataFrame με την ονομασία "Is_Closed", η οποία υποδεικνύει αν ένα περιστατικό θεωρείται κλειστό. Εάν το περιστατικό είναι κλειστό, η τιμή της στήλης είναι "1", διαφορετικά "0". Στη συνέχεια, εξάγουμε το έτος από τη στήλη "Date Rptd" (καθώς η δομή της πληροφορίας είναι MM/DD/YYYY) και το αποθηκεύουμε σε μια νέα στήλη, τη "Year". Κατόπιν, υπολογίζουμε το ποσοστό των κλειστών υποθέσεων ανά τομέα και έτος, χρησιμοποιώντας το "groupBy" σε συνδυασμό με τη συνάρτηση "agg" για τον υπολογισμό πλήθους ("count"). Δημιουργούμε ένα window για την κατάταξη των τομέων, ομαδοποιώντας τους ανά έτος και ταξινομώντας τους με βάση το ποσοστό επίλυσης περιστατικών σε φθίνουσα σειρά. Προσθέτουμε μια νέα στήλη "Rank", που περιέχει την κατάταξη κάθε τομέα, και φιλτράρουμε τα αποτελέσματα ώστε να διατηρηθούν οι τρεις κορυφαίοι τομείς ανά έτος. Τέλος, υπολογίζουμε τον χρόνο εκτέλεσης και εμφανίζουμε τα αποτελέσματα.
(τα αποτελέσματα εμφανίζονται στην επόμενη σελίδα)

Χρησιμοποιώντας SQL API:
Αρχικά, καταχωρούμε το DataFrame ως προσωρινό SQL πίνακα με όνομα "crime_data" και αποθηκεύουμε τον τρέχοντα χρόνο για τη μέτρηση της διάρκειας εκτέλεσης του SQL ερωτήματος. Στη συνέχεια, διαμορφώνουμε ένα SQL query που περιλαμβάνει τα εξής βήματα: εξαγωγή του έτους από τη στήλη "Date Rptd", υπολογισμός του ποσοστού των υποθέσεων που θεωρούνται "κλειστές", φθίνουσα κατάταξη των περιοχών με βάση το ποσοστό επίλυσης περιστατικών ανά έτος, φιλτράρισμα για τις τρεις κορυφαίες περιοχές κάθε έτους και τελική ταξινόμηση. Αφού εκτελεστεί το query, υπολογίζεται ο χρόνος εκτέλεσης και εμφανίζονται τα αποτελέσματα.

Σημείωση: Ένα περιστατικό θεωρείται κλειστό αν στην στήλη "Status Desc" δεν έχει την τιμή "UNK" ή "Invest Cont".

Αποτελέσματα για DataFrame

AREA NAME	Year	Closed_Case_Rate	Rank
Rampart	2010	32.947355855318136	1
Olympic	2010	31.962706191728422	2
Harbor	2010	29.63203463203463	3
Olympic	2011	35.212167689161554	1
Rampart	2011	32.511779630300836	2
Harbor	2011	28.652205202015008	3
Olympic	2012	34.414818310523835	1
Rampart	2012	32.9464181029429	2
Harbor	2012	29.815133276010318	3
Olympic	2013	33.52812271731191	1
Rampart	2013	32.08287360549221	2
Harbor	2013	29.16422459266206	3
Van Nuys	2014	31.80567315834039	1
West Valley	2014	31.31198995605775	2
Mission	2014	31.16279069767442	3
Van Nuys	2015	32.64134698172773	1
West Valley	2015	30.27597402597403	2
Mission	2015	30.179460678380153	3
Van Nuys	2016	31.880755720117726	1
West Valley	2016	31.54798761609907	2
Foothill	2016	29.870291843352458	3
Van Nuys	2017	32.02034211742949	1
Mission	2017	31.03892518634398	2
Foothill	2017	30.469226081657524	3
Foothill	2018	30.708950655075302	1
Mission	2018	30.690661478599225	2
Van Nuys	2018	29.078685730517943	3
West Valley	2019	30.77447195094254	1
Mission	2019	30.748519116855142	2
Foothill	2019	29.53842186694172	3
West Valley	2020	31.144886009717204	1
Mission	2020	30.38777908343126	2
Harbor	2020	29.880657509569918	3
Mission	2021	30.91391367253436	1
West Valley	2021	28.877503493246394	2
Foothill	2021	28.464788732394364	3
West Valley	2022	26.64366494153728	1
Harbor	2022	26.334056399132322	2
Topanga	2022	26.27340236376948	3
Foothill	2023	26.8215994531784	1
Topanga	2023	26.407806464728605	2
Mission	2023	25.941195616795053	3
N Hollywood	2024	19.514978601997147	1
Foothill	2024	18.531827515400412	2
77th Street	2024	17.349137931034484	3

DataFrame Implementation Time: 13.37 seconds

Αποτελέσματα για SQL

Year	Precinct	Closed_Case_Rate	Rank
2010	Rampart	32.947355855318136	1
2010	Olympic	31.962706191728422	2
2010	Harbor	29.63203463203463	3
2011	Olympic	35.212167689161554	1
2011	Rampart	32.511779630300836	2
2011	Harbor	28.652205202015008	3
2012	Olympic	34.414818310523835	1
2012	Rampart	32.9464181029429	2
2012	Harbor	29.815133276010318	3
2013	Olympic	33.52812271731191	1
2013	Rampart	32.08287360549221	2
2013	Harbor	29.16422459266206	3
2014	Van Nuys	31.80567315834039	1
2014	West Valley	31.31198995605775	2
2014	Mission	31.16279069767442	3
2015	Van Nuys	32.64134698172773	1
2015	West Valley	30.27597402597403	2
2015	Mission	30.179460678380153	3
2016	Van Nuys	31.880755720117726	1
2016	West Valley	31.54798761609907	2
2016	Foothill	29.870291843352458	3
2017	Van Nuys	32.02034211742949	1
2017	Mission	31.03892518634398	2
2017	Foothill	30.469226081657524	3
2018	Foothill	30.708950655075302	1
2018	Mission	30.690661478599225	2
2018	Van Nuys	29.078685730517943	3
2019	West Valley	30.77447195094254	1
2019	Mission	30.748519116855142	2
2019	Foothill	29.53842186694172	3
2020	West Valley	31.144886009717204	1
2020	Mission	30.38777908343126	2
2020	Harbor	29.880657509569918	3
2021	Mission	30.91391367253436	1
2021	West Valley	28.877503493246394	2
2021	Foothill	28.464788732394364	3
2022	West Valley	26.64366494153728	1
2022	Harbor	26.334056399132322	2
2022	Topanga	26.27340236376948	3
2023	Foothill	26.8215994531784	1
2023	Topanga	26.407806464728605	2
2023	Mission	25.941195616795053	3
2024	N Hollywood	19.514978601997147	1
2024	Foothill	18.531827515400412	2
2024	77th Street	17.349137931034484	3

SQL Implementation Time: 8.28 seconds

Αρχικά παρατηρούμε ότι τα αποτελέσματα και στις δύο περιπτώσεις είναι τα ίδια και ότι πιο γρήγορος είναι ο κώδικας με την SQL με μικρή διαφορά (5sec). Αυτό μπορεί να είναι χάρη στην πιο προηγμένη βελτιστοποίηση του Catalyst Optimizer και στην απλότητα του κώδικα της SQL υλοποίησης εν αντιθέσει του DataFrame που περιέχει πιο σύνθετες υλοποιήσεις στον κώδικά του. Το DataFrame API προσφέρει μεγαλύτερη ευχέρεια για δυναμικούς και σύνθετους υπολογισμούς, επιτρέποντας τη δημιουργία νέων στηλών, την εφαρμογή σύνθετων μετασχηματισμών και τη δυναμική επεξεργασία δεδομένων με την Python. Από την άλλη, το SQL API είναι πιο εύκολο στη σύνταξη αλλά είναι λιγότερο ευέλικτο όταν απαιτούνται πολύπλοκες μετατροπές ή αναλύσεις.

Parquet:

Ύστερα αποθηκεύουμε το dataset crime_data σε ένα αρχείο Parquet στο Amazon S3 και χρησιμοποιώντας DataFrame φορτώνουμε τα δεδομένα από δύο διαφορετικές πηγές, CSV και Parquet και μετράμε τον χρόνο φόρτωσης για κάθε περίπτωση.

Το αποτέλεσμα είναι το ακόλουθο:

```
CSV Load Time: 3.95 seconds
Parquet Load Time: 0.33 seconds
```

Ο χρόνος φόρτωσης των δεδομένων από το Parquet αναμενόταν να είναι σημαντικά μικρότερος σε σχέση με το CSV, κυρίως λόγω των διαφορών στη δομή και την αποδοτικότητα των δύο μορφών. Το Parquet είναι ένα columnar storage format, το οποίο επιτρέπει καλύτερη συμπίεση των δεδομένων και πιο αποδοτική αποθήκευση, μειώνοντας τον απαιτούμενο χρόνο για την ανάγνωση και την αποθήκευση τους.

Επιπλέον, το Parquet περιλαμβάνει μεταδεδομένα και πληροφορίες για τους τύπους των δεδομένων, επιτρέποντας στον Apache Spark να διαχειρίζεται τα δεδομένα πιο γρήγορα και χωρίς την ανάγκη ανάλυσης τύπων. Το CSV, από την άλλη πλευρά, είναι μια απλή μορφή χωρίς συμπίεση ή βελτιστοποιήσεις, και απαιτεί περισσότερο χρόνο για τη φόρτωση δεδομένων, καθώς πρέπει να αναγνωρίζονται οι τύποι και να διαβάζονται όλες οι γραμμές του αρχείου, ακόμη και αν δεν χρειάζονται για την ανάλυση.

Στην συνέχεια, εκτελούμε έναν αντίστοιχο κώδικα με προηγουμένως για το συγκεκριμένο query κάνοντας χρήση DataFrame όπου όμως η διαφορά αφορά την προέλευση των δεδομένων όπου τώρα είναι Parquet.

Ο δεύτερος κώδικας είναι πιθανό να εκτελείται γρηγορότερα, καθώς χρησιμοποιεί Parquet, που είναι βελτιστοποιημένη μορφή δεδομένων για μεγάλους όγκους.

Τα αποτελέσματα είναι τα ακόλουθα:

AREA NAME	Year	Closed_Case_Rate	Rank
Rampart	2010	32.947355855318136	1
Olympic	2010	31.962706191728422	2
Harbor	2010	29.63203463203463	3
Olympic	2011	35.212167689161554	1
Rampart	2011	32.511779630300836	2
Harbor	2011	28.652205202015008	3
Olympic	2012	34.414818310523835	1
Rampart	2012	32.9464181029429	2
Harbor	2012	29.815133276010318	3
Olympic	2013	33.52812271731191	1
Rampart	2013	32.08287360549221	2
Harbor	2013	29.16422459266206	3
Van Nuys	2014	31.80567315834039	1
West Valley	2014	31.31198995605775	2
Mission	2014	31.16279069767442	3
Van Nuys	2015	32.64134698172773	1
West Valley	2015	30.27597402597403	2
Mission	2015	30.179460678380153	3
Van Nuys	2016	31.880755720117726	1
West Valley	2016	31.54798761609907	2
Foothill	2016	29.870291843352458	3
Van Nuys	2017	32.02034211742949	1
Mission	2017	31.03892518634398	2
Foothill	2017	30.469226081657524	3
Foothill	2018	30.708950655075302	1
Mission	2018	30.690661478599225	2
Van Nuys	2018	29.078685730517943	3
West Valley	2019	30.77447195094254	1
Mission	2019	30.748519116855142	2
Foothill	2019	29.53842186694172	3
West Valley	2020	31.144886009717204	1
Mission	2020	30.38777908343126	2
Harbor	2020	29.880657509569918	3
Mission	2021	30.91391367253436	1
West Valley	2021	28.877503493246394	2
Foothill	2021	28.464788732394364	3
West Valley	2022	26.64366494153728	1
Harbor	2022	26.334056399132322	2
Topanga	2022	26.27340236376948	3
Foothill	2023	26.8215994531784	1
Topanga	2023	26.407806464728605	2
Mission	2023	25.941195616795053	3
N Hollywood	2024	19.514978601997147	1
Foothill	2024	18.531827515400412	2
77th Street	2024	17.349137931034484	3

DataFrame (Parquet) Implementation Time: 3.30 seconds

Πράγματι όταν τα δεδομένα είναι σε μορφή Parquet το query εκτελείται πιο γρήγορα.

- **Query 3**

Υλοποιούμε το query αυτό χρησιμοποιώντας DataFrame, στο query αυτό χρησιμοποιώντας τα δεδομένα απογραφή για τον πληθυσμό του 2010 και τα δεδομένα απογραφής για το εισόδημα του 2015 υπολογίζουμε για κάθε περιοχή του Los Angeles το μέσο ετήσιο εισόδημα ανά άτομο και την αναλογία συνολικού αριθμού εγκλημάτων ανά άτομο.

Αρχικά, φορτώνουμε τα απαραίτητα δεδομένα για το query, που περιλαμβάνουν δύο αρχεία .csv με πληροφορίες εγκλημάτων και ένα .csv με δεδομένα εισοδημάτων.
Γενικά επεξεργαζόμαστε τρία σύνολα δεδομένων (πληθυσμός απογραφής, εισόδημα νοικοκυριού και εγκλήματα) για να υπολογίσουμε μετρήσεις για κάθε περιοχή του Λος Άντζελες (οι περιοχές ορίζονται από την στήλη COMM των δεδομένων απογραφής). Πρώτον, φορτώνουμε τα δεδομένα απογραφής, συγκεντρώνουμε τον πληθυσμό κατά ταχυδρομικό κώδικα (στήλη ZCTA10) και αντιστοιχίζουμε τους ταχυδρομικούς κώδικες σε περιοχές.
Στη συνέχεια, τα income data φιλτράρονται για κοινότητες στο Los Angeles (city= Los Angeles). Τα δεδομένα εγκλήματος ενώνονται γεωχωρικά με τις γεωμετρίες ταχυδρομικού κώδικα που φτιάξαμε ωστε να αντιστοιχιστούν τα εγκλήματα σε ταχυδρομικούς κώδικες, και μετά υπολογίζουμε τα συνολικών εγκλήματα για κάθε ταχυδρομικό κώδικα. Τα δεδομένα πληθυσμού και εισοδήματος ενώνονται με ταχυδρομικούς κώδικες και το σύνολο δεδομένων που προκύπτει ενώνεται με τα crime data. Τα αποτελέσματα μαζεύονται σε έναν πίνακα που δείχνει τις μετρήσεις

Zip codes

Αρχικά δημιουργούμε τις γεωμετρίες για τα δεδομένα εγκλημάτων.
Χρησιμοποιώντας τις στήλες LAT και LON, δημιουργούμε γεωγραφικά σημεία (με τη χρήση του ST_Point) για κάθε έγκλημα. Κανουμε aggregate όλες τις γεωμετρίες για κάθε ταχυδρομικό κώδικα (στήλη ZCTA10) χρησιμοποιώντας το ST_Union_Aggr και για κάθε έγκλημα, βρίσκουμε τον ταχυδρομικό κώδικα στον οποίο ανήκει ελέγχοντας εάν η γεωμετρία του εγκλήματος (geom) είναι εντός της γεωμετρίας του μπλοκ απογραφής. Έπειτα, μετράμε τον συνολικό αριθμό των εγκλημάτων για κάθε ταχυδρομικό κώδικα και αντιστοιχίζουμε τους ταχυδρομικούς κώδικες σε περιοχές.

Χρησιμοποιούμε τη στήλη COMM από τα δεδομένα της απογραφής για να αντιστοιχίσουμε ταχυδρομικούς κώδικες σε περιοχές του Λος Άντζελες και διασφαλίζουμε distinct αντιστοιχίσεις για την αποφυγή αντιγράφων. Τέλος, ενώνουμε τα εγκλήματα με τις περιοχές συνδυάζοντας τα συνολικά εγκλήματα (κατά ταχυδρομικό κώδικα) με τις περιοχές από τα δεδομένα απογραφής.

Στη συνέχεια, γίνονται τα απαραίτητα join για τη συνολική ανάλυση των δεδομένων. Πρώτα, συνδέονται τα crime data με τα population δεδομένα και με τα εισοδήματα. Τέλος, όλα τα δεδομένα (πληθυσμός, εισόδημα, εγκλήματα) συνδυάζονται σε ένα τελικό DataFrame, όπου υπολογίζονται βασικοί δείκτες, όπως το μέσο εισόδημα ανά άτομο και ο αριθμός εγκλημάτων ανά άτομο για κάθε περιοχή.

Ο πίνακας που παράγουμε είναι ο εξής

AREA NAME	Avg_Income_Per_Person	Crimes_Per_Person
Harbor	1.4934860695460446	0.0566400113534792
N Hollywood	1.3563236944348895	0.11537073481563351
Van Nuys	1.912627858121802	0.12540546833246022
West Valley	1.7451880929912311	0.10015834679547672
Wilshire	1.5595552388254463	0.0876788768698825
Topanga	1.8691518914045722	0.11288243232127776
Foothill	1.6931473127605596	0.09648812181490662
Devonshire	1.4373115668557885	0.10632421404220281
Pacific	1.5193721551315027	0.0457701780115032
Central	1.2982976174407537	0.07162145829528843
West LA	2.518749265028811	0.10631103260752178
Northeast	1.4655195533922605	0.08425209474024864
Mission	1.457785754352768	0.09859339630219838
Southeast	0.978406514446836	0.07837444443200792
Olympic	1.6105674771799032	0.10683180560652891
Southwest	1.2897477408777571	0.08648157490560059
Rampart	1.3334276857649796	0.0938562759325357
77th Street	1.0924050632911393	0.1178531736759015
Hollywood	1.6293112264614198	0.11762010228513836
Newton	1.1768343293492696	0.0869503207825973

Κατά το “explain” του βασικού query:
Η πρώτη φάση είναι το **Parsed Logical Plan** όπου εδώ εμφανίζεται η λογική του query πριν εφαρμοστούν οι optimizers.
Η δεύτερη φάση είναι το **Analyzed Logical Plan** όπου εκεί επαληθεύεται η ορθότητα του query.
Η τρίτη φάση είναι το **Optimized Logical Plan** όπου εφαρμόζονται οι optimizers.
Τέλος η τέταρτη φάση είναι το **Physical Plan** που εμφανίζει πως ακριβώς θα υλοποιηθεί το query.

Από την έξοδο του explain στο βασικό query φαίνεται ότι το πρώτο join είναι το BroadcastJoin και το δεύτερο είναι SortMergeJoin.

BROADCAST:
Σε αυτή τη μέθοδο ο ένας από τους δύο πίνακες του join στέλνεται με broadcast messages σε όλους τους κόμβους και ο δεύτερος πίνακας γίνεται partitioned και στέλνεται κατά τμήματα σε διαφορετικούς κόμβους. Χρησιμοποιείται όταν ένα από τα σύνολα δεδομένών είναι αρκετά μικρό ώστε να χωράει στη μνήμη κάθε κόμβου εργασίας. Είναι ιδανικό για το join ενός μεγάλου συνόλου δεδομένων με ένα μικρό. Επίσης μειώνει την ανάγκη για ανακάτεμα των δεδομένων στους κόμβους, άρα η εκτέλεση είναι πιο γρήγορη. Εξαιρετικά γρήγορο για μικρά δεδομένα, καθώς δεν απαιτείται shuffle.

MERGE:
Πρώτα γίνει η διαδικασία του shuffling (οι δύο πίνακες κατανέμονται κατάλληλα μεταξύ των executors, records και των δύο πινάκων με συγκεκριμένο value του join key πηγαίνουν στον ίδιο κόμβο), στη συνέχεια ο κάθε κόμβος εσωτερικά εκτελεί sort merge join, οι πίνακες ταξινομούνται βάσει του join key και στη συνέχεια γίνεται το matching των records μέσω του merge. Είναι αποτελεσματικό για μεγάλα σύνολα δεδομένων όπου και οι δύο πλευρές είναι πολύ μεγάλες για να χωρέσουν στην μνήμη. Απαιτεί ανακάτεμα των δεδομένων, αλλά είναι αποδοτικό όσον αφορά τη χρήση μνήμης. Είναι καλή πρακτική για μεγάλα, ανακατεμένα join σε δεδομένα που δεν αξιοποιούν το broadcast join. Αποδοτικό για μεγάλα δεδομένα που είναι ταξινομημένα ή μπορούν να ταξινομηθούν.

SHUFFLE_HASH:
Η μέθοδος επίσης θα ξεκινήσει με shuffling, όπου αναφέρθηκε και παραπάνω, στην συνέχεια εντός κάθε κόμβου γίνεται hash join μεταξύ των τμημάτων των δύο πινάκων που έχει κάθε executor. Η μέθοδος αυτή ενδείκνυται σε περιπτώσεις αρκετά μεγάλων πινάκων που γίνονται τα joins και δεν θα ήταν εφικτό το broadcast join αλλά και αρκετά μικρά για να μπορεί να δημιουργηθεί ένα hash table για την μία πλευρά του join. Καλή επιλογή για μεσαίου μεγέθους δεδομένα, αλλά μπορεί να επηρεαστεί από μη ομοιόμορφη κατανομή των δεδομένων.

SHUFFLE_REPLICATE_NL:
Εκτελείται join μεταξύ δύο datasets, A και B, μέσω κατανομής τους σε partitions. Όλα τα partitions του B αποστέλλονται σε κάθε partition του A, ώστε κάθε executor να εκτελεί nested loop join. Δηλαδή, για κάθε εγγραφή του A ελέγχεται η αντιστοιχία με όλες τις εγγραφές όλων των partitions του B, υλοποιώντας έτσι ένα εξαντλητικό cartesian join. Χρησιμοποιείται για καρτεσιανές συνδέσεις όπου κάθε γραμμή ενός συνόλου δεδομένών συνδέεται με κάθε γραμμή ενός άλλου συνόλου δεδομένών. Πολύ απαιτητική σε πόρους και συνήθως δεν συνιστάται.
Πολύ αργό και χρησιμοποιείται μόνο ως έσχατη λύση.

Συμπεράσματα

Τέλος, αναγκάσαμε το Spark να χρησιμοποιήσει διαφορετικές στρατηγικές (μεταξύ των BROADCAST, MERGE, SHUFFLE_HASH, SHUFFLE_REPLICATE_NL) και τα αποτελέσματα που πήραμε είναι τα εξής:

Join_Tested	Strategy	Time
1st Join	BROADCAST	9.323999881744385
1st Join	MERGE	9.470892667770386
1st Join	SHUFFLE_HASH	9.460222721099854
1st Join	SHUFFLE_REPLICATE_NL	9.613125324249268

Join_Tested	Strategy	Time
2nd Join	BROADCAST	805.8332898616791
2nd Join	MERGE	562.7038254737854
2nd Join	SHUFFLE_HASH	923.6534628868103
2nd Join	SHUFFLE_REPLICATE_NL	619.0023169517517

Τα αποτελέσματα δείχνουν μια έντονη διαφοροποίηση στους χρόνους εκτέλεσης μεταξύ του πρώτου και του δεύτερου join.

Το πρώτο join (population + income) είναι αρκετά γρήγορο σε όλες τις στρατηγικές, διαρκεί περίπου 9,3 έως 9,6 δευτερόλεπτα ανεξάρτητα από τον τύπο. Αυτό δείχνει ότι τα μεγέθη των datasets που κάνουν join είναι σχετικά μικρά, με αποτέλεσμα να έχει καλή απόδοση ακόμη και το "SHUFFLE_REPLICATE_NL" (που είναι συνήθως το πιο αργό).

Ωστόσο, το δεύτερο join (αποτέλεσμα πρώτου join + crime data) διαρκεί πολύ περισσότερο, με χρόνους που κυμαίνονται από 562 έως 923 δευτερόλεπτα, υποδεικνύοντας ότι αυτό το join είναι υπολογιστικά ακριβό λόγω του μεγάλου μεγέθους των δεδομένων εγκλήματος (περίπου 3,1 εκατομμύρια rows). Η σύνδεση MERGE είναι η ταχύτερη για το δεύτερο join (562s), κάτι που είναι λογικό δεδομένου ότι τα Sort-Merge join έχουν πολύ καλή απόδοση κατά τον χειρισμό μεγάλων συνόλων δεδομένων. Οι ενώσεις SHUFFLE_HASH και BROADCAST είναι αρκετά πιο αργές, με το BROADCAST να παίρνει 805s, πιθανότατα λόγω της υπερβολικής επιβάρυνσης της μνήμης. Το SHUFFLE_HASH έχει τη χειρότερη απόδοση (923s). Παραδόξως, το SHUFFLE_REPLICATE_NL (619s) είναι ταχύτερο από το BROADCAST και το SHUFFLE_HASH, πιθανώς λόγω του Spark που βελτιστοποιεί μικρότερα partitions αντί να κάνει ένα πλήρως τυχαίο shuffle. Συνολικά, η καλύτερη στρατηγική φαίνεται να είναι ένας συνδυασμός BROADCAST

για την πρώτη ένωση και MERGE για τη δεύτερη σύνδεση, καθώς εξισορροπεί την αποτελεσματικότητα τόσο σε μικρά όσο και σε μεγάλα σύνολα δεδομένων.

Το γεγονός ότι οι στρατηγικές join με την καλύτερη απόδοση ήταν και αυτές που επιλέχθηκαν από τον Catalyst Optimizer του Spark δείχνει ότι ο optimizer λαμβάνει σωστές αποφάσεις με βάση τα χαρακτηριστικά δεδομένων, τη δομή των queries και τους διαθέσιμους πόρους αφού γενικά έχει σχεδιαστεί για να επιλέγει αυτόματα την πιο αποτελεσματική στρατηγική join.

- **Query 4**

Υλοποιούμε το query αυτό χρησιμοποιώντας DataFrame, στο query αυτό βρίσκουμε το φυλετικό προφίλ των καταγεγραμμένων θυμάτων των εγκλημάτων στο Los Angeles για το 2015 στις 3 περιοχές με το υψηλότερο αλλά και με το χαμηλότερο κατά κεφαλήν εισόδημα.

Εκτελέσαμε την υλοποίηση εφαρμόζοντας κλιμάκωση στο σύνολο των υπολογιστικών πόρων. Χρησιμοποιώντας 2 executors και τα ακόλουθα:
• 1core/2 GB memory • 2cores/4GB memory • 4cores/8GB memory

Αρχικά, φορτώνουμε τα δεδομένα που απαιτούνται για το query, τα οποία περιλαμβάνουν δύο αρχεία .csv με δεδομένα εγκλημάτων, το .csv με τα δεδομένα εισοδημάτων, το αρχείο με τους κωδικούς φυλής και εθνικότητας, καθώς και το GeoJSON αρχείο με τα δεδομένα των περιοχών. Όπως και στο query 3, προετοιμάζουμε τα δεδομένα του πληθυσμού από το GeoJSON αρχείο με την χρήση του Sedona. Στη συνέχεια, συνδυάζουμε τα πληθυσμιακά και εισοδηματικά δεδομένα για περιοχές του Los Angeles και υπολογίζουμε το ανά κεφαλήν εισόδημα. Εντοπίζουμε τις 3 περιοχές με το υψηλότερο και τις 3 με το χαμηλότερο, χρησιμοποιώντας τις συναρτήσεις "orderBy" και "limit". Φιλτράρουμε τα δεδομένα για να απομονώσουμε τα εγκλήματα του 2015, έπειτα όμοια με προηγούμενος φτιάχνουμε τα Zip Codes στα δεδομένα των εγκλημάτων και εφαρμόζουμε γεωχωρικό join με τη συνάρτηση ST_Within. Τέλος, συγχωνεύουμε τα δεδομένα εγκλημάτων με τα φυλετικά δεδομένα και ομαδοποιούμε, υπολογίζοντας τον αριθμό των θυμάτων για κάθε φυλετική ομάδα. Παρουσιάζουμε τα αποτελέσματα για τις περιοχές με το υψηλότερο και το χαμηλότερο κατά κεφαλήν εισόδημα.
Τα αποτελέσματα:

```
Top areas racial profile
+--------------------+-------------+
|   Vict Descent Full|Total_Victims|
+--------------------+-------------+
|               White|          272|
|               Other|           97|
|Hispanic/Latin/Me...|           45|
|               Black|           29|
|             Unknown|           20|
|         Other Asian|           17|
|            Filipino|            1|
+--------------------+-------------+
```

```
Bottom areas racial profile
+--------------------+-------------+
|   Vict Descent Full|Total_Victims|
+--------------------+-------------+
|Hispanic/Latin/Me...|         7239|
|               Black|         4572|
|               Other|          412|
|               White|          363|
|         Other Asian|           69|
|             Unknown|           54|
|            Filipino|            3|
|            Japanese|            2|
|             Chinese|            2|
|              Korean|            2|
|           Guamanian|            1|
|American Indian/A...|            1|
+--------------------+-------------+
```

Για διαφορετικά configurations:

```
2 executors 2g memory 1 core: 359.22 seconds
2 executors 4g memory 2 core: 181.92 seconds
2 executors 8g memory 4 core: 164.34 seconds
```

Τα αποτελέσματα δείχνουν ότι η αύξηση της μνήμης και των πυρήνων ανά executor βελτιώνει σημαντικά τον χρόνο εκτέλεσης, αλλά η βελτίωση μειώνεται όσο αυξάνεται η διαθέσιμη υπολογιστική ισχύς. Συγκεκριμένα, η μετάβαση από 2g μνήμη/1 core σε 4g μνήμη/2 cores μείωσε τον χρόνο κατά περίπου 49%, ενώ η αύξηση σε 8g μνήμη/4 cores προσέφερε οριακή βελτίωση (περίπου 1%). Αυτό υποδηλώνει ότι το σύστημα φτάνει σε σημείο κορεσμού, όπου η επιπλέον ισχύς δεν αξιοποιείται πλήρως.

- **Query 5**

Υλοποιούμε το query αυτό χρησιμοποιώντας DataFrame, στο query αυτό υπολογίσαμε ανά αστυνομικό τμήμα, τον αριθμό των εγκλημάτων που έλαβαν χώρα πλησιέστερα σε αυτό, καθώς και τη μέση απόστασή του από τις τοποθεσίες όπου σημειώθηκαν τα συγκεκριμένα περιστατικά.

Εκτελέσαμε την υλοποίηση σε διάφορες διαμορφώσεις αξιοποιώντας συνολικά 8 πυρήνες επεξεργαστή και 14GB μνήμης. Ο κώδικας παρέμενε ο ίδιος, εμείς αλλάζαμε μόνο τους πόρους. Η λογική του κώδικα παρουσιάζεται παρακάτω:

Στην αρχή φορτώνουμε τα δεδομένα που θα χρησιμοποιήσουμε για αυτό το query τα οποία είναι τα δύο .csv για τα εγκλήματα, το .csv για τα αστυνομικά τμήματα και το census_blocks .geojson. Με την βοήθεια του SedonaContext, διαβάζουμε το .geojson αρχείο για τη διαχείριση και ανάλυση γεωχωρικών δεδομένων. Δημιουργούμε γεωμετρικές στήλες για σημεία εγκλημάτων αλλά και αστυνομικών τμημάτων, υπολογίζουμε τις αποστάσεις μεταξύ τους μέσω του ST_DistanceSphere το οποίο λαμβάνει υπόψιν την σφαιρική γεωμετρία της γης και εμφανίζουμε τα αποτελέσματα (άρα για κάθε έγκλημα υπολογίζεται η απόσταση του από κάθε αστυνομικό τμήμα). Έπειτα δημιουργούμε για κάθε έγκλημα ένα window που περιέχει τις αποστάσεις από τα αστυνομικά τμήματα και τα ταξινομούμε με βάση την απόσταση αυτή. Με την χρήση της row_number() αναθέτουμε σε μία στήλη "rank" τον αριθμό κατάταξης για κάθε απόσταση και κρατάμε αυτό που έχει rank 1, έτσι έχουμε αντιστοιχίσει το κάθε έγκλημα στο κοντινότερο αστυνομικό τμήμα. Τέλος υπολογίζουμε τον συνολικό αριθμό εγκλημάτων με χρήση του "count" και την μέση απόσταση με χρήση του "avg" για κάθε αστυνομικό τμήμα.

Τα αποτελέσματα είναι τα ακόλουθα:

DIVISION	Average_Distance	Number_of_Crimes
HOLLYWOOD	2076.2639601787287	224340
VAN NUYS	2953.3697428197916	210134
SOUTHWEST	2191.398805780876	188901
WILSHIRE	2592.665532978778	185996
77TH STREET	1716.5449719701087	171827
OLYMPIC	1723.603697178097	170897
NORTH HOLLYWOOD	2643.006094141568	167854
PACIFIC	3850.070655307911	161359
CENTRAL	992.4764374568804	153871
RAMPART	1534.5341879190169	152736
SOUTHEAST	2421.866215888182	152176
WEST VALLEY	3035.67121631406	138643
TOPANGA	3296.9548417555843	138217
FOOTHILL	4250.921708425002	134896
HARBOR	3702.5615993564984	126747
HOLLENBECK	366921.3044614887	119294
WEST LOS ANGELES	2792.457289034109	115781
NEWTON	1634.635739709743	111110
NORTHEAST	3623.6655246040796	108109
MISSION	3690.942614278604	103355

Για διαφορετικά configurations:

- 2 executors × 4 cores/8GB memory

```
2 executors 8g memory 4 core: 43.96 seconds
```

- 4 executors × 2 cores/4GB memory

```
4 executors 4g memory 2 core: 21.06 seconds
```

- 8 executors × 1 core/2 GB memory

```
8 executors 2g memory 1 core: 16.92 seconds
```

Τα αποτελέσματα αυτά μας δείχνουν ότι η υψηλή παραλληλοποίηση παίζει σημαντικό ρόλο στη μείωση του χρόνου εκτέλεσης. Παρότι η διαμόρφωση με 2 executors είχε περισσότερη μνήμη και πυρήνες ανά executor, ο περιορισμένος αριθμός executors οδήγησε σε σημαντικά μεγαλύτερο χρόνο (43.96s). Αντίθετα, η διαμόρφωση με 8 executors και μικρότερη μνήμη (2GB) και 1 core ανά executor ήταν η ταχύτερη (16.92s), λόγω αυξημένης παραλληλοποίησης. Η ενδιάμεση διαμόρφωση με 4 executors πέτυχε καλή ισορροπία (21.06s), δείχνοντας ότι περισσότερος αριθμός executors, ακόμα και με μικρότερη μνήμη, μπορεί να είναι πιο αποτελεσματικός όταν ο κώδικας δεν απαιτεί υπερβολική μνήμη για κάθε εργασία.